

Filmtár Nyilvántartó Rendszer – Specifikáció

1. A feladat általános ismertetése

A projekt célja egy olyan asztali alkalmazás készítése C++ nyelven, amely képes különböző típusú filmek (játékfilmek, dokumentumfilmek, családi filmek) adatainak rendszerezett tárolására, kezelésére és tartós mentésére. A program egy heterogén kollekcióban tárolja az adatokat, lehetővé téve a különböző filmtípusok egységes kezelését.

2. Funkcionális követelmények

2.1. Adatkezelés

- **Új film felvétele:** A felhasználó megadhatja a film alapadatait (cím, évszám, hossz).
- **Film törlése:** Meglévő rekord eltávolítása a nyilvántartásból cím vagy azonosító alapján.
- **Módosítás:** Egy már felvett film adatainak felülírása.

2.2. Listázás és keresés

- **Teljes listázás:** Az összes tárolt film adatainak megjelenítése.
- **Szűrt keresés:** Keresési lehetőség cím, kiadási év vagy kategória szerint.
- **Rendezés:** A lista rendezése évszám vagy cím szerint.

2.3. Fájlkezelés

- **Mentés fájlba:** Parancsra az adatok szöveges fájlba (pl. filmek.txt) írása.
- **Betöltés fájlból:** Indításkor a korábban elmentett adatok visszaolvasása.

3. Fogalmi modell és osztályszerkezet

3.1. Osztályhierarchia

- **Film (Absztrakt ősz osztály):** Tárolja a közös attribútumokat: cím (változó méretű szöveg) ◦ hossz (percekben) ◦ év (kiadás éve)
- **CsaladiFilm (Származtatott):** Extra attribútum: korhatar.
- **Dokumentumfilm (Származtatott):** Extra attribútum: leiras (rövid összefoglaló).

3.2. Adatszerkezet

- **Filmtar osztály:** A heterogén kollekciót kezelő osztály. ◦ adatok (tömb). ◦ db, kapacitas: A dinamikus tömb méretének kezeléséhez. ◦ Metódusok a hozzáadáshoz, törléshez és a fájlkezeléshez.

3.3. Segédosztályok

- **String osztály:** Az STL std::string kiváltására.

4. Felhasználói felület (UI)

A program karakteres felületen (TUI) kommunikál. A főmenü pontjai:

1. Filmek listázása
2. Keresés cím alapján
3. Új film hozzáadása (típusválasztással)
4. Film törlése
5. Adatok mentése és kilépés

5. Implementációs korlátok és elvárások

- **Memóriakezelés**
- **Modularitás:** A kódot külön .h és .cpp fájlokra kell bontani.
- **Hordozhatóság:** A kódnak szabványos C++11 környezetben, külső könyvtárak nélkül kell fordulnia.

6. Adattárolás módja (Példa)

A fájlban az adatokat soronként, elválasztó karakterrel (pl. vessző vagy pontosvessző) tároljuk:

T;Cím;Hossz;Év;Extra_adat Példa:

C;Oroszlánkirály;88;1994;6

D;Természet ereje;50;2020;Vulkánokról szóló film

7.1. Fájlkezelési hibák

- **Hiányzó adatfájl:** Amennyiben a filmek.txt nem található, a program nem állhat le hibával. Ilyenkor üres adatbázissal indul el és ezt jelzi a felhasználónak. Az első mentéskor a program létrehozza az új fájlt.
- **Hibás fájlstruktúra:** Ha a fájl létezik, de a tartalma nem felel meg a specifikációnak (pl. hiányzó mezők, sérült karakterek), a betöltés az adott sornál megszakad. A már beolvasott, érvényes adatokat meg kell tartani, a hibás sorról pedig tájékoztatni kell a felhasználót.
- **Írási jogosultság:** Ha a mentés során a fájl nem írható (pl. jogosultsági probléma), a program figyelmezteti a felhasználót, és lehetőséget ad a mentés megismétlésére egy másik fájl névvel vagy egy másik helyre.

7.2. Felhasználói bevétel kezelése

- **Érvénytelen adattípusok:** Ha a menüválasztásnál vagy számérték megadásánál (pl. évszám, játékidő) a felhasználó nem számot ad meg, a programnak újra be kell kérnie az adatot.
- **Tartományellenőrzés:** A program ellenőrzi a bevitt adatok logikai helyességét. Például negatív lejátszási idő vagy a jövőre vonatkozó (pl. 2100-as) kiadási év nem fogadható el.

- **Túl hosszú karakterláncok:** A saját String osztálynak kezelnie kell a tetszőleges hosszúságú bemenetet.

7.3. Dinamikus memóriakezelés

- **Foglalási hiba:** A memóiafoglalás sikertelensége esetén a programnak el kell kapnia a kivételt vagy ellenőriznie kell a pointert, majd rendezett módon (az eddigi adatok mentésével) le kell állnia.
- **Indexelés:** A heterogén kollekció elérésekor minden esetben ellenőrizni kell, hogy az adott index a tömb határain belül esik-e.

7.4. Kivételkezelés

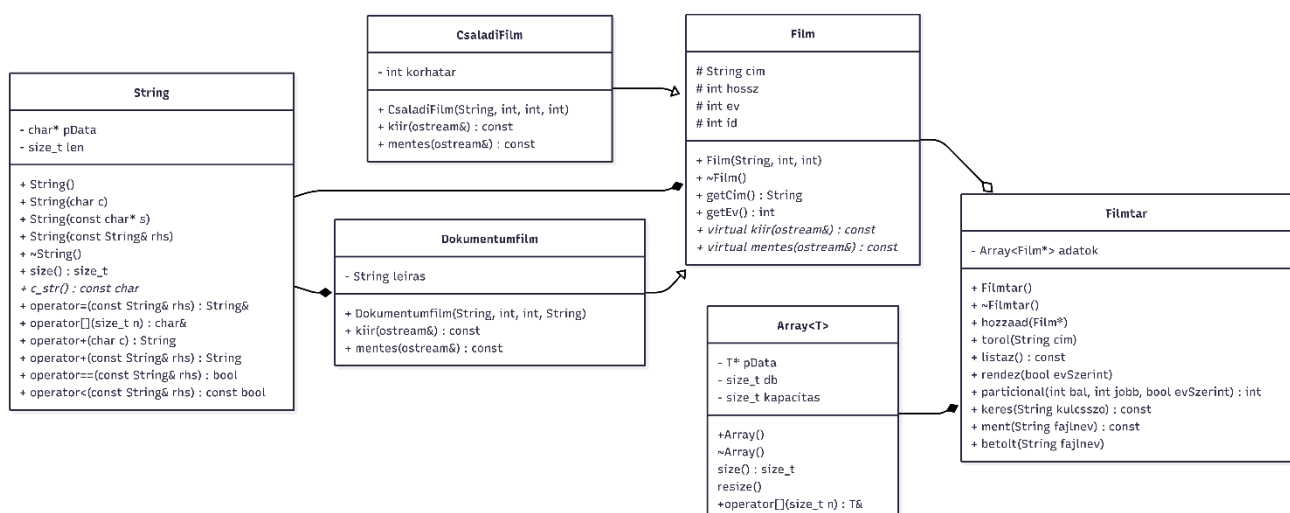
A kritikus pontokon (fájl megnyitása, memória foglalása, típuskonverzió) figyelni kell a kivételeket. A saját osztályok (pl. Filmtar) dobjanak egyedi hibakódokat, ha olyan állapotba kerülnek, amelyet nem tudnak megoldani.

8. Pontosított feladat-specifikáció

A projekt célja egy C++11 szabvány szerinti asztali alkalmazás készítése, amely különböző típusú filmek (játékfilmek, dokumentumfilmek, családi filmek) adatait kezeli. A rendszernek támogatnia kell:

- **Heterogén kollekciót:** Különböző filmtípusok egységes tárolása dinamikus tömbben, absztrakt őssztyálon keresztül.
- **Perzisztens adattárolást:** Az adatok mentése és betöltése szöveges fájlból (CSV-szerű formátum).
- **Alapműveleteket:** Új film felvétele, törlése, módosítása, keresése és rendezése.
- **Saját segédosztályok:** Az STL könyvtár helyett saját String osztály és dinamikus memóriakezelés alkalmazása.
- **Hibaellenőrzést:** Felhasználói bevitel ellenőrzése (típus és tartomány) és fájlkezelési hibák lekezelése.

9. UML diagram



10. Osztályok felelőssége és memóriakezelés

10.1. Logika osztályai

- **Film (Absztrakt ősosztály):** Meghatározza a közös interfészt és tárolja az alapvető attribútumokat (cím, hossz percben, év). A destruktora virtuális a helyes memóriafelszabadítás érdekében.
- **Származtatott osztályok:** A CsaladiFilm és Dokumentumfilm osztályok kiegészítik az ősosztályt a specifikus adatokkal (korhatár, leírás) és megvalósítják a virtuális kiíró metódusokat.
- **Filmtar:** A heterogén kollekció gazdája. Felelős a dinamikus Film* pointereket tartalmazó tömb kezeléseért, a memóiafoglalásért és a felszabadításért.
- **A Filmtar adattagja egy sablonos tömb class lesz, ami a tömbműveleteket kezeli, ezáltal átláthatóbb/bővithetőbb a kód**

10.2. Segédosztályok és infrastruktúra

- **String:** Felelős a dinamikus szövegtárolásért, kiváltva az std::string-et.
- **Memóriakezelés:** Dinamikus tömb a gyors elérés miatt. A Filmtar destruktora nemcsak a pointereket tartalmazó tömböt, hanem a rajta keresztül elért összes Film objektumot is törli a memóriából.

11. Fontosabb algoritmusok

11.1. Új elem hozzáadása és dinamikus átméretezés

Amennyiben a db == kapacitas, a tömböt meg kell növelni:

1. Új, kétszeres méretű ideiglenes tömb foglalása.
2. A régi pointerok átmásolása az új tömbbe.
3. Az új elem elhelyezése az első szabad helyen, db növelése.
4. A régi tömb felszabadítása.
5. A tömb pointerét átállítjuk a temp pointerre.

11.2. Fájlból való betöltés

A program soronként olvassa a fájlt:

IF sor első karaktere == 'C':

Paraméterek beolvasása (vesszők mentén) -> Új CsaladiFilm objektum foglalása.

ELSE IF sor első karaktere == 'D':

Paraméterek beolvasása -> Új Dokumentumfilm objektum foglalása.

Filmtar.hozzaad(uj_objektum);

Hiba esetén (pl. sérült sor) a program tájékoztatja a felhasználót, de a már beolvasott adatokat megőrzi.

11.3. Keresés és rendezés

Lineáris keresést alkalmazunk a heterogén kollekción. A felhasználó által megadott kulcsszót összehasonlítjuk az objektumok cím attribútumával a `String::operator==` segítségével.

A `rendez()` egy gyorsrendező algoritmust fog használni, ennek a segédfüggvénye a `particional()`

```
Void Rendez(int bal, int jobb, bool mód) {
    IF bal < jobb
        int p = particional(bal, jobb, mód)
        Rendez (bal, p-1)
        Rendez(p+1, jobb)
}

Int Particional(int bal, int jobb, bool mód) {
    Valasztó_elem = adatok[jobb]
    I = bal -1

    CIKLUS j = bal -tol jobb-1 -ig
        IF (mód == CÍM ÉS adatok[j]->getCim() < választó_elem->getCim()) VAGY (mód == ÉV
            ÉS adatok[j]->getEv() < választó_elem->getEv())

            i = i + 1 CSERE(adatok[i], adatok[j]) // csak a pointereket cseréljük
    CIKLUS VÉGE
    CSERE(adatok[i + 1], adatok[jobb])
    VISSZAAD i + 1
}
```

12. Felhasználói felület (TUI) elkülönítése

A logika (Filmtar) nem tartalmaz cin vagy cout hívásokat a mentés/listázás metódusokon kívül. A beviteli adatokat a main függvényben ellenőrizzük (pl. számformátum, tartományellenőrzés), és csak a validált adatokat adjuk át a programnak.